

3.

Laporan Tugas Praktikum 10



Muhammad Bintang Gunawan - 0110224003

Teknik Informatika, STT Terpadu Nurul Fikri, Depok

0110224003@student.nurulfikri.ac.id

4.

Abstract

penerapan algoritma *K-Nearest Neighbor* (KNN) dalam tiga studi kasus: klasifikasi manual suhu udara, evaluasi performa model kelulusan, dan prediksi musim berbasis data cuaca menggunakan Python.

1. Latihan 1 : Klasifikasi Suhu Udara Menggunakan KNN

Langkah-langkah analisis yang dilakukan :

- **Identifikasi Variabel:** Menentukan variabel independen (fitur), yaitu Temperatur x dan Kecepatan Angin y, serta variabel dependen (target) yaitu Klasifikasi (Dingin/Panas).
- **Data Uji :** Data yang akan diklasifikasikan adalah kondisi dengan Temperatur 16 C dan Kecepatan Angin 3 km/jam.
- **Ranking :** Mengurutkan hasil perhitungan jarak dari yang terkecil hingga terbesar untuk menemukan tetangga terdekat (*nearest neighbors*).

Temperatur	Kecepatan An	Klasifikasi	Jarak Euclid	Rank	Data Uji	udara	kecepatan
10	0	Dingin	6,708203932	5		16	3
25	0	Panas	9,486832981	8			
15	5	Dingin	2,236067977	1		hasil k neighbo	3
20	3	Panas	4	2			dingin
18	7	Dingin	4,472135955	3			
20	10	Dingin	8,062257748	6			
22	5	Panas	6,32455532	4			
24	6	Panas	8,544003745	7			

Kesimpulan Akhir

Berdasarkan analisis menggunakan algoritma KNN dengan $k=3$, dapat disimpulkan bahwa pada kondisi temperatur udara 16C dengan kecepatan angin 3 km/jam, persepsi Marry terhadap suhu udara tersebut adalah DINGIN.

5.

2. Latihan 2: Evaluasi Model

Langkah-langkah Analisis

- **Komparasi Data:** Ini adalah langkah mencocokkan hasil tebakan model (*Predicted*) dengan kenyataan lapangan (*Actual*), dilakukan per mahasiswa (NIM TI001 sampai TI010). Tujuannya sederhana: melihat prediksi mana yang benar dan mana yang salah.

- **Pembuatan Confusion Matrix:**

Mengelompokkan hasil perbandingan ke dalam 4 kategori

1. TP (True Positive)
2. TN (True Negative)
3. FP (False Positive)
4. FN (False Negative)

- **Perhitungan Metrik:** Menghitung nilai *Accuracy*, *Precision*, dan *Recall* berdasarkan nilai-nilai dalam *Confusion Matrix*.

6.

Nim	Hasil Sebenarnya (Actual)	Hasil Prediksi (Predicted)		
TI001	Lulus	Lulus		
TI002	Lulus	Lulus		
TI003	Lulus	Lulus		
TI004	Lulus	Tidak Lulus		
TI005	Lulus	Tidak Lulus		
TI006	Tidak Lulus	Lulus		
TI007	Tidak Lulus	Tidak Lulus		
TI008	Tidak Lulus	Tidak Lulus		
TI009	Tidak Lulus	Tidak Lulus		
TI010	Tidak Lulus	Tidak Lulus		
Total	10	10		
Confusion Matrix				
	Predicted: Lulus (Positif)	Predicted: Tidak Lulus (Negatif)	Total Sebenarnya	
Actual: Lulus (Positif)	3	2	5	
Actual: Tidak Lulus (Negatif)	1	4	5	
Total Prediksi	4	6	Total	10
Metrik	hasil	Hasil Persentase		
Accuracy	0,7	70%		
Precision	0,75	75%		
Recall	0,6	60%		

Kesimpulan

Model prediksi ini memiliki tingkat akurasi yang cukup baik (70%). Namun, nilai Recall lebih rendah (60%) dibandingkan Precision (75%) menunjukkan bahwa model ini agak "pelit" dalam memprediksi kelulusan; model cenderung melewati beberapa mahasiswa yang sebenarnya lulus (terdeteksi sebagai *False Negative*).

3. Studi kasus Prediksi Cuaca menggunakan algoritma KNN

1.Data Preprocessing

7.

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import GridSearchCV, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from imblearn.over_sampling import SMOTE
from collections import Counter

```

Loading dataset

```

from google.colab import drive
drive.mount('/content/drive')

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

df_cuaca = pd.read_csv('/content/drive/MyDrive/PRAKTIKUM ML/Praktikum10/Data/weather_classification_data.csv')

```

Menampilkan data awal

```

df_cuaca.head()

```

	Temperature	Humidity	Wind Speed	Precipitation (%)	Cloud Cover	Atmospheric Pressure	UV Index	Season	Visibility (km)	Location	Weather Type
0	14.0	73	9.5	82.0	partly cloudy	1010.82	2	Winter	3.5	inland	Rainy
1	39.0	96	8.5	71.0	partly cloudy	1011.43	7	Spring	10.0	inland	Cloudy
2	30.0	64	7.0	16.0	clear	1018.72	5	Spring	5.5	mountain	Sunny
3	38.0	83	1.5	82.0	clear	1026.25	7	Spring	1.0	coastal	Sunny
4	27.0	74	17.0	66.0	overcast	990.67	1	Winter	2.5	mountain	Rainy

```

print("Informasi Dataset:")
df_cuaca.info()

```

```

... Informasi Dataset:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 13200 entries, 0 to 13199
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Temperature            13200 non-null  float64
1   Humidity                13200 non-null  int64
2   Wind Speed              13200 non-null  float64
3   Precipitation (%)       13200 non-null  float64
4   Cloud Cover             13200 non-null  object
5   Atmospheric Pressure    13200 non-null  float64
6   UV Index                13200 non-null  int64
7   Season                  13200 non-null  object
8   Visibility (km)         13200 non-null  float64
9   Location                13200 non-null  object
10  Weather Type            13200 non-null  object
dtypes: float64(5), int64(2), object(4)
memory usage: 1.1+ MB

```

2. Data Cleaning

8.

```
1 print(df_cuaca['Cloud Cover'].unique())
  print(df_cuaca['Season'].unique())
  print(df_cuaca['Location'].unique())
  print(df_cuaca['Weather Type'].unique())

['partly cloudy' 'clear' 'overcast' 'cloudy']
['Winter' 'Spring' 'Summer' 'Autumn']
['inland' 'mountain' 'coastal']
['Rainy' 'Cloudy' 'Sunny' 'Snowy']
```

Menghapus Kolom yang Tidak Digunakan

```
1 df_cuaca.drop(columns=['Cloud Cover'],inplace=True)
```

+ Kode + Teks

```
replacement = {
    'Season': {
        'Winter': 0,
        'Spring': 1,
        'Summer': 2,
        'Autumn': 3
    },
    'Location': {
        'coastal': 0,
        'inland': 1,
        'mountain': 2
    },
    'Weather Type': {
        'Sunny': 0,
        'Cloudy': 1,
        'Rainy': 2,
        'Snowy': 3
    }
}

df_cuaca = df_cuaca.replace(replacement, inplace=False)
df_cuaca.head()
```

/tmp/ipython-input-2843386111.py:21: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version. To retain
df_cuaca = df_cuaca.replace(replacement, inplace=False)

	Temperature	Humidity	Wind Speed	Precipitation (%)	Atmospheric Pressure	UV Index	Season	Visibility (km)	Location	Weather Type
0	14.0	73	9.5	82.0	1010.82	2	0	3.5	1	2
1	39.0	96	8.5	71.0	1011.43	7	1	10.0	1	1
2	30.0	64	7.0	16.0	1018.72	5	1	5.5	2	0
3	38.0	83	1.5	82.0	1026.25	7	1	1.0	0	0
4	27.0	74	17.0	66.0	990.67	1	0	2.5	2	2

Melakukan Encoding pada Dataset Uji

```
df_cuaca = df_cuaca.replace(replacement, inplace=False)
df_cuaca.head()
```

```
***
```

	Temperature	Humidity	Wind Speed	Precipitation (%)	Atmospheric Pressure	UV Index	Season	Visibility (km)	Location	Weather Type
0	14.0	73	9.5	82.0	1010.82	2	0	3.5	1	2
1	39.0	96	8.5	71.0	1011.43	7	1	10.0	1	1
2	30.0	64	7.0	16.0	1018.72	5	1	5.5	2	0
3	38.0	83	1.5	82.0	1026.25	7	1	1.0	0	0
4	27.0	74	17.0	66.0	990.67	1	0	2.5	2	2

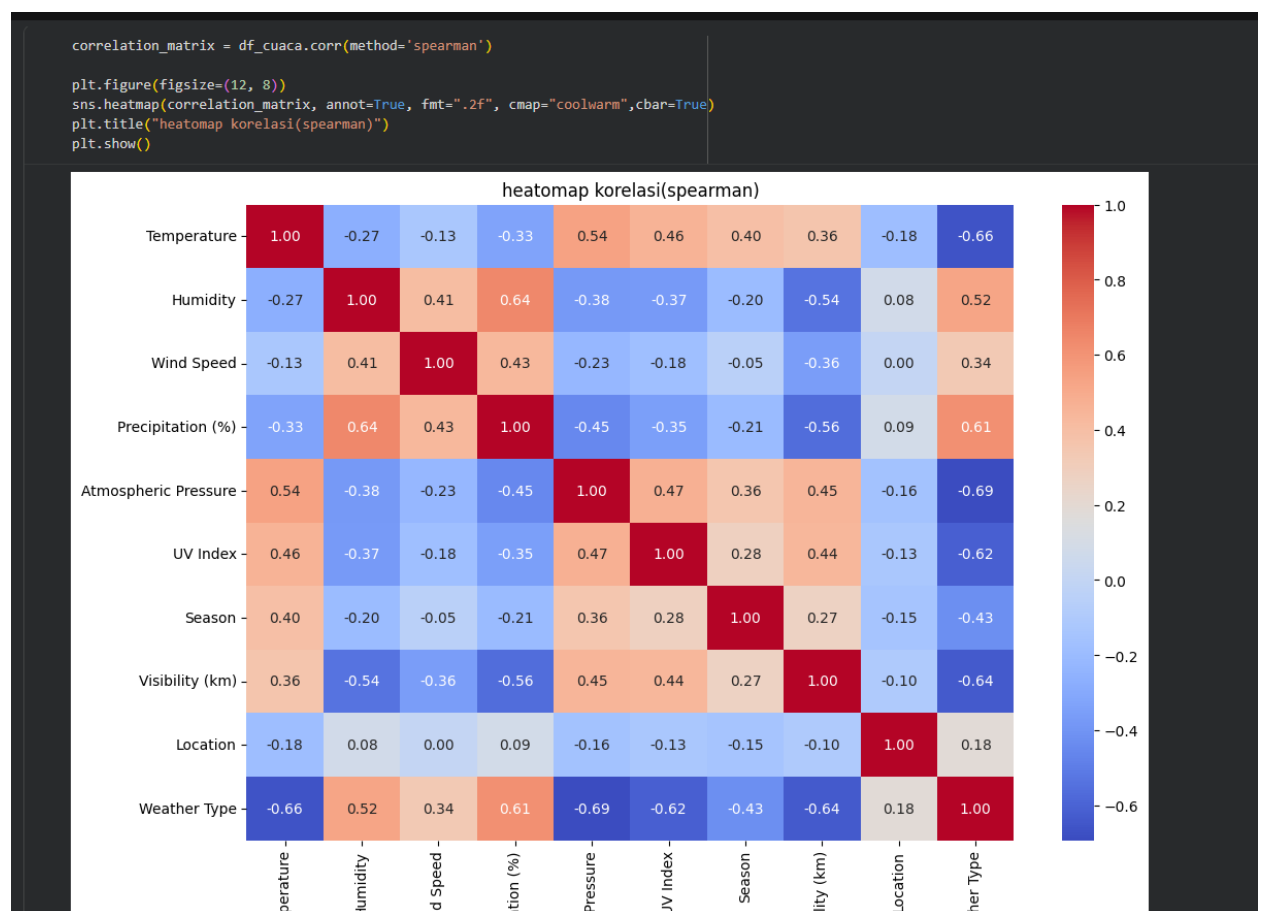
9.

```
df_cuaca.isnull().sum()

...
0
Temperature 0
Humidity 0
Wind Speed 0
Precipitation (%) 0
Atmospheric Pressure 0
UV Index 0
Season 0
Visibility (km) 0
Location 0
Weather Type 0

dtype: int64
```

3. Analisis Korelasi Fitur



10.



4. Pemisahan data & Normalisasi

```
X_cuaca = df_cuaca.drop(columns=['Season'])
y_cuaca = df_cuaca['Season']
scaler = StandardScaler()
X_cuaca_scaled = scaler.fit_transform(X_cuaca)
```

Menangani Ketidakseimbangan dengan SMOTE

```
print("BEFORE SMOTE")
print(y_cuaca.value_counts())

sm = SMOTE(random_state=42,k_neighbors=1)
X_train_smote, y_train_smote = sm.fit_resample(X_cuaca_scaled, y_cuaca)
print("AFTER SMOTE")
print(y_train_smote.value_counts())
```

BEFORE SMOTE

```
Season
0    5610
1    2598
3    2500
2    2492
Name: count, dtype: int64
```

AFTER SMOTE

```
Season
0    5610
1    5610
2    5610
3    5610
Name: count, dtype: int64
```

11.

5. Menentukan Parameter Terbaik

```
param_grid = {
    'n_neighbors': list(range(1, 31)),
    'weights': ['uniform', 'distance'],
    'metric': ['euclidean', 'manhattan']
}
grid = GridSearchCV(KNeighborsClassifier(), param_grid, cv=5, n_jobs=-1)
grid.fit(X_train_smote, y_train_smote)

print("Best Parameters:", grid.best_params_)
print("Best Score:", grid.best_score_)

... Best Parameters: {'metric': 'euclidean', 'n_neighbors': 1, 'weights': 'uniform'}
Best Score: 0.8354723707664885
```

6. Melatih Model KNN

```
best_knn = grid.best_estimator_
best_knn.fit(X_cuaca_scaled, y_cuaca)

y_pred = best_knn.predict(X_cuaca_scaled)
```

Evaluasi Model dengan Classification Report

```
print("\nClassification Reports:")
print(classification_report(y_cuaca, y_pred))
```

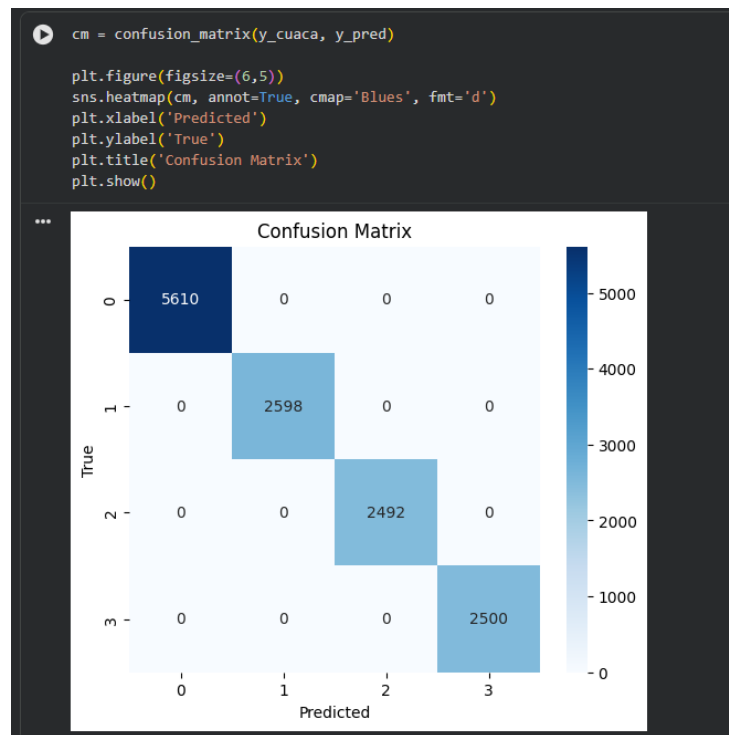
```
...
Classification Reports:
              precision    recall  f1-score   support

     0           1.00        1.00        1.00        5610
     1           1.00        1.00        1.00        2598
     2           1.00        1.00        1.00        2492
     3           1.00        1.00        1.00        2500

 accuracy          1.00
 macro avg          1.00
 weighted avg       1.00
```

12.

7. Evaluasi dengan Confusion Matrix



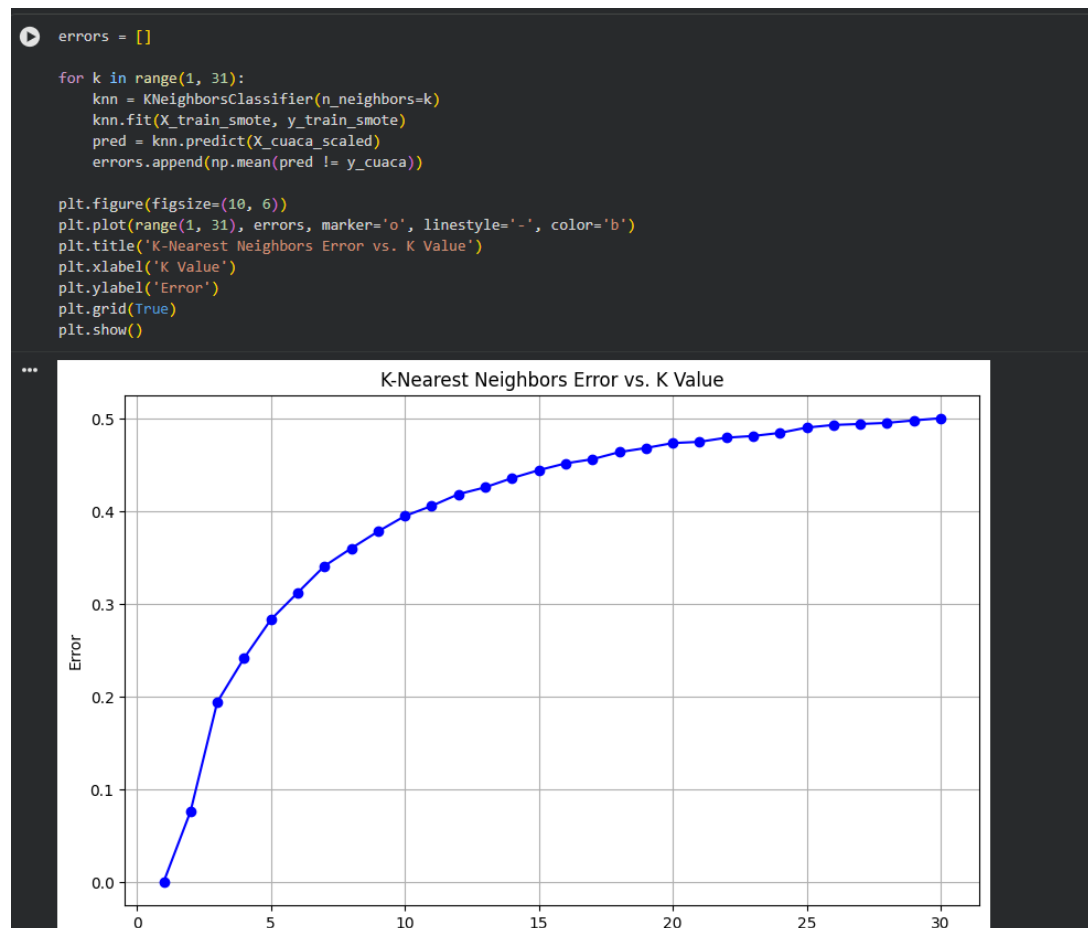
8. Validasi Model dengan Cross Validation

```
cv_scores = cross_val_score(best_knn, X_train_smote, y_train_smote, cv=5)
print("Cross-Validation Scores:", cv_scores.mean())
print("Std Dev:", cv_scores.std())
```

*** Cross-Validation Scores: 0.8354723707664885
Std Dev: 0.06217218004434773

13.

9. Menentukan Nilai K Optimal dengan Elbow Method



10. Kesimpulan

Penerapan algoritma KNN yang dikombinasikan dengan SMOTE dan *GridSearch* pada studi kasus ini menghasilkan model yang sangat presisi. Penyeimbangan data menggunakan SMOTE berhasil mengatasi bias pada data mayoritas (*Winter*).

14.

Link GitHub:

https://github.com/XiaoMao15/ML_Praktikum-dan-Tugas.git